TOTALISATION

Réconciliation avec le produit logiciel

Un manifeste philosophique pour ceux qui créent, maintiennent et transcendent le logiciel

PARTIE B

par **Léo Michel Bélanger**

ORIGINALIUM | ŢŢĻŞŢ~

© Léo Michel Bélanger - Tous droits réservés
Images et couverture par Soléane Bélanger
Publié par ORIGINALIUM | ŢŢĻŞŢ~

ŢŢĻŞŢ~ - Les deux dimensions fondamentales

Dimension 1 - Substantif linguistique : « Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser. »

Dimension 2 - Philosophie ORIGINALIUM : « Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption. »

Note de transition - De la Partie A à la Partie B

Dans la Partie A de ce manifeste, tu as été invité à revoir les fondements mêmes de ce que signifie "faire du logiciel" sous l'angle de la Totalisation. Tu as exploré la naissance du concept ŢŢĻŞŢ~, compris pourquoi la réconciliation avec le produit logiciel n'est pas un vœu pieux mais une nécessité philosophique et pratique, et traversé les quatre premiers chapitres qui ont posé les assises de cette vision : la reconnaissance du produit comme entité vivante, la réconciliation comme posture, le rôle de l'intention dans la création logicielle, et la nature de la relation entre l'humain et son produit.

La Partie B t'amène maintenant plus profondément dans le concret philosophique. Elle touche au cycle de vie, à la qualité, aux parties prenantes, à l'architecture et à l'identité. Elle te demande de regarder non plus seulement *pourquoi* tu construis un logiciel, mais *comment* la Totalisation transforme chaque dimension de cette construction. Lis ce qui suit comme tu lirais une lettre de quelqu'un qui a traversé les mêmes doutes que toi - et qui en est revenu, transformé.

Le cycle de vie du produit vu par Totalisation

« Un produit logiciel ne commence pas quand quelqu'un tape la première ligne de code. Il commence quand quelqu'un ressent, pour la première fois, que quelque chose devrait exister. »

Le produit logiciel ne naît pas à la première ligne de code

Il y a une illusion tenace dans le monde du développement logiciel - une illusion si profondément ancrée qu'elle structure toute la manière dont les équipes pensent, organisent et mesurent leur travail. Cette illusion, c'est celle du commencement technique. On croit, collectivement, que le produit logiciel prend naissance au moment précis où un développeur ouvre son éditeur de code, crée un nouveau fichier, et tape les premières instructions qui vont finir par constituer ce produit.

C'est faux. Et cette fausseté n'est pas anodine. Elle est à l'origine de certaines des plus grandes fractures que l'on observe dans les équipes de développement : la fracture entre les gens qui "imaginent" et ceux qui "exécutent", la fracture entre la vision et l'implémentation, la fracture entre ce que le produit devrait être et ce qu'il devient réellement.

La Totalisation - ŢŢĻŞŢ~ - refuse cette illusion avec une clarté absolue. Selon la philosophie ORIGINALIUM, le produit logiciel naît au moment où quelqu'un, quelque part, ressent un manque. Un manque réel, concret, souffrant. Ce sentiment - cette douleur d'absence - est le premier souffle du produit. Tout ce qui vient après, la conception, le code, le déploiement, la maintenance, n'est que la prolongation de ce premier souffle dans le monde matériel.

Comprendre ceci change absolument tout dans la manière dont tu abordes ton travail. Si tu commences à compter à partir de la première ligne de code, tu

coupes la vie du produit en deux : tu ignores sa gestation, tout ce qui a précédé cette première ligne et qui en explique chaque choix. Tu deviens alors un exécutant sans mémoire. La Totalisation te demande d'être un gardien avec une mémoire complète - avant, pendant et après.

TTLSTˆ~ - Principe fondamental

Le cycle de vie d'un produit logiciel commence au moment où l'intention de sa création émerge dans la conscience humaine. La Totalisation reconnaît et honore cette origine invisible autant que le code visible qui en découle. Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption.

La conception comme premier souffle du produit

Si la naissance du produit réside dans le ressenti d'un manque, alors la conception est véritablement son premier souffle - le moment où ce manque prend forme, où il commence à s'articuler en intentions, en contours, en possibles. La conception n'est pas un prérequis bureaucratique au développement. Elle est une phase de vie à part entière, aussi importante que n'importe quelle autre.

Or, dans beaucoup d'organisations, la conception est traitée comme un couloir. On la traverse le plus vite possible pour arriver à l'"essentiel" : le code. Des décisions fondamentales y sont prises en quelques jours - parfois en quelques heures - sans qu'on prenne le temps de les totaliser, c'est-à-dire de les considérer dans leur ensemble, dans leur relation avec tout ce qui précède et tout ce qui suivra. Résultat : des produits qui portent, dans leur architecture même, les cicatrices de ces décisions bâclées.

La Totalisation te propose une autre relation à la conception. Elle te dit : traite la conception comme tu traiterais la naissance d'un être vivant. Donne-lui le temps

et l'attention qu'elle mérite. Pose les questions difficiles maintenant, pendant que tout est encore malléable, pendant que le produit n'est encore que potentiel. Quels sont ses valeurs fondamentales ? Pour qui existe-t-il vraiment ? Qu'est-ce qu'il refusera catégoriquement de faire, même si certains utilisateurs le demandent ? Quelle est l'identité qu'il portera dans vingt ans, si il vit jusque-là ?

Ces questions ne sont pas philosophiques dans le sens "inutile" du terme. Ce sont des questions de survie du produit. Les produits qui n'ont pas été questionnés en profondeur lors de leur conception sont ceux qui dérivent, qui perdent leur identité, qui finissent par ne plus savoir pourquoi ils existent. La conception totalisante est une conception qui embrasse l'ensemble de la trajectoire du produit, même quand cette trajectoire est encore inconnue.

L'évolution continue : le produit vivant

Un produit logiciel qui ne change pas est un produit qui meurt. Ce n'est pas une métaphore - c'est une réalité observée dans chaque industrie, dans chaque contexte, à chaque époque de l'histoire du logiciel. Le monde change. Les utilisateurs changent. Les technologies changent. Les besoins évoluent. Et un produit qui refuse de s'inscrire dans cette évolution finit invariablement par se retrouver en décalage avec la réalité qu'il est censé servir.

Mais l'évolution n'est pas synonyme d'agitation. Beaucoup d'équipes confondent les deux. Elles changent constamment - de fonctionnalités, d'interfaces, d'architectures - sans que ces changements s'inscrivent dans une direction cohérente. C'est du mouvement sans sens, de l'activité sans orientation. La Totalisation fait une distinction capitale entre ces deux états.

L'évolution vraie, au sens de T̰T̰L̰S̰T̰~, est une évolution qui reste fidèle à l'identité profonde du produit tout en s'adaptant aux nouvelles réalités. C'est l'arbre qui grandit, dont les branches s'étendent dans de nouvelles directions, mais dont les racines restent fermement ancrées dans le même sol. Chaque nouvelle version,

chaque nouveau module, chaque nouvelle fonctionnalité devrait pouvoir répondre à cette question simple : "Comment ce changement exprime-t-il et approfondit-il ce que ce produit est fondamentalement ?"

Si tu ne peux pas répondre à cette question, c'est peut-être le signe que le changement que tu t'apprêtes à faire n'est pas une évolution - c'est une déviation. Et la Totalisation te demande d'être assez courageux pour nommer cette différence et d'agir en conséquence.

La retraite du logiciel : quand un produit meurt dignement

Il y a une vérité que personne n'aime aborder dans les organisations technologiques : les produits logiciels ont une fin. Comme tout être vivant, ils naissent, évoluent, vieillissent, et finissent par céder la place à autre chose. La Totalisation ne fuit pas cette vérité. Elle l'embrasse, parce qu'une philosophie qui ne peut pas regarder la fin en face ne peut pas non plus regarder honnêtement le début.

Mais la manière dont un produit se retire du monde dit beaucoup sur la culture de l'organisation qui l'a créé. Un produit qui meurt dans l'abandon - sans documentation de transition, sans plan de migration pour les utilisateurs, sans reconnaissance de ce qu'il a accompli - c'est une organisation qui n'a jamais vraiment aimé ce qu'elle créait. Elle a utilisé le produit, elle l'a exploité, mais elle ne l'a pas respecté.

La retraite digne d'un produit logiciel, selon ŢŢĻŞŢ~, commence bien avant la fin officielle. Elle commence quand on reconnaît honnêtement que le produit a accompli sa mission et qu'un successeur, interne ou externe, est mieux placé pour continuer le service. Elle implique une passation de connaissance rigoureuse - non seulement des instructions techniques, mais aussi du *pourquoi* : pourquoi

certaines décisions ont été prises, quels compromis ont été acceptés et dans quel contexte, quelles leçons ont été apprises au fil des années de vie du produit.

Une organisation qui sait faire mourir dignement ses produits est une organisation qui sait aussi les faire naître et vivre dignement. Ce sont les deux faces d'une même sagesse totalisante.

Totalisation et la continuité de vie du produit

La grande contribution de la philosophie ŢŢĻŞŢ~ au cycle de vie du produit logiciel, c'est peut-être cette idée de continuité. Non pas la continuité technique - la capacité d'un système à rester en ligne - mais la continuité de sens. La capacité d'un produit à rester fidèle à lui-même à travers toutes les phases de sa vie, à conserver un fil rouge qui relie son origine à chacune de ses versions successives.

Cette continuité ne s'entretient pas seule. Elle demande un travail actif, constant, de la part de toutes les personnes impliquées dans la vie du produit. Elle demande que chaque décision soit évaluée non seulement à l'aune de ses effets immédiats, mais aussi dans sa relation avec l'ensemble du cycle de vie : "Est-ce que ce choix que je fais aujourd'hui va enrichir ou appauvrir ce que ce produit est et sera ?" La Totalisation, c'est exactement cette question posée en permanence - avant, pendant et après chaque geste.

CHAPITRE 6

La qualité du produit selon ŢŢĻŞŢ~

« La qualité n'est pas ce que tu mesures après coup. C'est ce que tu décides d'être avant de commencer. »

Définir la qualité logicielle : au-delà des métriques

Demande à dix professionnels du logiciel ce que signifie "qualité", et tu obtiendras dix réponses différentes. Pour certains, c'est l'absence de bugs. Pour d'autres, c'est la performance, la couverture de tests, le respect des normes ISO, le score de satisfaction utilisateur, ou encore la capacité à évoluer sans tout casser. Chacune de ces réponses est vraie. Et pourtant, aucune d'elles, prise isolément, ne capture la totalité de ce qu'est la qualité.

C'est précisément là que la Totalisation intervient. La philosophie ŢŢĻṢŢ~ ne rejette aucune de ces définitions - elle les rassemble. Elle te dit que la qualité est la synthèse de toutes ces dimensions, considérées ensemble, dans leur relation mutuelle et dans leur rapport à l'identité et à la mission du produit. La qualité totalisée n'est pas une liste de critères à cocher. C'est une posture, une orientation, une intention qui traverse l'ensemble du cycle de vie du produit.

Et c'est ici que réside la première révolution que ŢŢĻṢŢ~ propose dans la manière de penser la qualité : passer d'une conception additive de la qualité - la somme de critères isolés - à une conception intégrée, où chaque attribut de qualité n'a de sens que dans sa relation avec les autres et avec la mission globale du produit.

Définition ŢŢĻṢŢ~ - Qualité totalisée

La qualité d'un produit logiciel, selon la philosophie ORIGINALIUM, est la capacité de ce produit à exprimer de manière cohérente, complète et continue son intention fondamentale - en servant ses utilisateurs, en respectant ses créateurs, en s'inscrivant dans son contexte et en demeurant fidèle à son identité à travers le temps. Elle est avant, pendant et après, en continu, sans interruption.

La qualité comme intention, pas comme résultat

La manière la plus commune de parler de qualité dans le développement logiciel, c'est de la mesurer après coup. On attend que le produit soit construit, puis on le teste, on le mesure, on détecte les défauts, on les corrige. Ce modèle - qui est encore dominant dans une grande partie de l'industrie - traite la qualité comme un résultat à vérifier. Et c'est là son problème fondamental.

Quand tu traites la qualité comme un résultat, tu acceptes implicitement la possibilité que le produit soit de mauvaise qualité pendant sa construction. Tu acceptes que la mauvaise qualité soit un état temporaire qui sera "corrigé" plus tard. Et cette acceptation, même implicite, crée une culture où la qualité est secondaire à la vitesse, où "on verra après" devient une réponse acceptable à des questions de fond.

La Totalisation renverse complètement cette logique. Elle te dit que la qualité est une intention qui doit précéder chaque geste, chaque décision, chaque ligne de code. Elle n'est pas quelque chose qu'on ajoute à la fin - c'est quelque chose qu'on choisit au début et qu'on maintient tout au long. "Qu'est-ce que cela donne comme qualité ?" devrait être la première question posée avant chaque choix de conception, chaque architecture décidée, chaque fonctionnalité développée.

Cette philosophie a des implications profondes. Elle signifie que la qualité n'appartient pas seulement à l'équipe de test - elle appartient à tout le monde. Le concepteur qui bâcle un flux utilisateur parce qu'il est pressé fait une chose de mauvaise qualité. Le développeur qui écrit du code difficile à lire "pour aller plus vite" crée de la mauvaise qualité. L'architecte qui accumule de la dette technique sans plan de remboursement détruit de la qualité. La Totalisation rend chacun responsable de la qualité dans son propre domaine d'action, avant, pendant et après.

Les attributs de qualité vus par Totalisation

La philosophie ŢŢĻŞŢ~ ne réinvente pas les attributs de qualité logicielle reconnus par l'industrie - elle les réinterprète à travers le prisme de la Totalisation. Voici comment les principaux attributs se transforment sous ce regard :

Attribut de qualité	Vision conventionnelle	Vision ŢŢĻŞŢ~
Fiabilité	Le système fonctionne sans pannes dans des conditions normales.	Le système honore sa promesse envers l'utilisateur en tout temps - pas seulement en conditions normales, mais dans la totalité des situations prévisibles et dans la gestion digne des situations imprévues.
Maintenabilité	Le code peut être modifié sans trop d'effort.	Le produit est conçu pour accueillir le changement comme une partie naturelle de sa vie, sans trahir son identité fondamentale. La maintenabilité est l'expression de la bienveillance envers les créateurs futurs.
Utilisabilité	L'interface est intuitive et facile à prendre en main.	Le produit respecte profondément l'intelligence et le temps de l'utilisateur. L'utilisabilité n'est pas un attribut de surface - c'est l'expression de la relation que le produit entretient avec ceux qu'il sert.
Performance	Le système répond rapidement et utilise les ressources de manière efficace.	Le produit accomplit sa mission sans gaspiller les ressources - ni les ressources techniques, ni le temps de

Attribut de qualité	Vision conventionnelle	Vision ŢŢĻŞŢ~
		l'utilisateur, ni les investissements de l'organisation. La performance est une forme de respect.
Sécurité	Le système protège les données et les accès contre les menaces.	Le produit assume la responsabilité totale de la confiance que lui accordent ses utilisateurs. La sécurité est une promesse faite à l'ensemble de la chaîne humaine qui touche au produit.
Portabilité	Le système peut être déployé dans différents environnements.	Le produit reste utile à travers les évolutions de son contexte technologique. La portabilité est une forme de résilience et d'humilité : on reconnaît que l'environnement changera.

La qualité intrinsèque vs qualité extrinsèque revisitées

La distinction classique entre qualité intrinsèque et qualité extrinsèque - ce que le produit est en lui-même versus ce que les utilisateurs perçoivent de lui - est réelle et utile. Mais la Totalisation refuse de la traiter comme une opposition. Elle la voit comme une tension productive à reconcilier.

La qualité intrinsèque sans qualité extrinsèque est un produit techniquement excellent que personne ne veut utiliser. La qualité extrinsèque sans qualité intrinsèque est un produit séduisant qui s'effondre à la première épreuve réelle. Ni l'un ni l'autre ne représente la qualité au sens de ŢŢĻŞŢ~.

La Totalisation te propose d'abandonner cette dichotomie et de la remplacer par une vision intégrée : la qualité d'un produit est la cohérence entre ce qu'il est profondément et ce qu'il exprime vers l'extérieur. Un produit de haute qualité selon ŢŢĻŞŢ~ est un produit dont l'excellence interne se reflète naturellement dans l'expérience qu'il procure - et dont l'excellence externe est soutenue par une solidité interne réelle. Les deux dimensions ne se trahissent pas l'une l'autre. Elles se renforcent, se totalisant dans une expression unifiée.

Comment BEASTEST mesure la maturité de la qualité du produit

La méthode BEASTEST, avec ses sept niveaux, offre un cadre de lecture de la maturité de la qualité d'un produit logiciel qui va bien au-delà des simples métriques techniques. Chaque niveau représente une profondeur de compréhension et d'intégration de la qualité dans la culture du produit.

Méthode BEASTEST - Les 7 niveaux de maturité de la qualité

Niveau

Couleur

Maturité de la qualité selon ŢŢĻŞŢ~

1

ROUGE

La qualité est absente ou ignorée. Le produit fonctionne "par accident". Les défauts sont découverts par les utilisateurs en production.

2

ORANGE

La qualité est réactive. On corrige les bugs après qu'ils sont signalés. Il n'y a pas

de processus de prévention systématique.

3

JAUNE

La qualité est proactive mais fragmentée. Des pratiques de test existent, mais elles ne sont pas intégrées dans une vision d'ensemble du produit.

4

VERT

La qualité est intégrée dans le processus de développement. Des standards existent et sont respectés. La qualité commence à être une intention partagée.

5

BLEU

La qualité est une culture. Toute l'équipe se sent responsable. Les attributs de qualité sont discutés et équilibrés dans chaque décision importante.

6

INDIGO

La qualité est totalisante. Elle traverse tous les niveaux du cycle de vie - conception, développement, déploiement, évolution - de manière cohérente et continue.

7

VIOLET

La qualité est l'identité. Le produit ne peut pas être pensé sans elle. Elle est l'expression de valeurs profondes qui définissent ce que l'organisation est et aspire à être.

Remarque que la progression de ROUGE à VIOLET n'est pas seulement une progression technique - c'est une progression philosophique. Les premiers niveaux parlent de *ce qu'on fait* avec la qualité. Les derniers niveaux parlent de *ce qu'on est* en relation à la qualité. La Totalisation aspire au niveau VIOLET : non pas parce qu'il est le plus difficile à atteindre, mais parce qu'il est le seul niveau où la qualité devient vraiment inséparable de l'identité du produit et de ceux qui le créent.

CHAPITRE 7

Le produit et ses parties prenantes : une relation totale

« Personne ne possède vraiment un produit logiciel. Tout le monde en est responsable. Et c'est précisément cette tension entre propriété et responsabilité que la Totalisation vient réconcilier. »

Qui "possède" vraiment le produit logiciel ?

La question de la propriété d'un produit logiciel est l'une des plus conflictuelles dans les organisations technologiques. Et pour cause : elle touche à des enjeux profonds de pouvoir, de responsabilité, de reconnaissance et d'identité. Selon les contextes, tu entendras des réponses très différentes à cette question.

Le chef de produit dira : "C'est moi qui possède le produit - je définis la vision, je priorise le backlog, je suis responsable du succès." Le directeur technique dira : "Sans l'équipe de développement, le produit n'existerait pas - ce sont eux les véritables propriétaires." Le directeur commercial dira : "C'est l'organisation qui possède le produit - elle a investi les ressources pour le créer." L'utilisateur, s'il

était consulté, dirait peut-être : "C'est *moi* qui possède ce produit - je l'utilise chaque jour, il fait partie de mon quotidien."

Toutes ces réponses contiennent une part de vérité. Et toutes sont insuffisantes prises isolément. La Totalisation propose une réponse différente, plus profonde et plus honnête : personne ne possède le produit logiciel au sens absolu du terme. Le produit est une création collective, portée par une communauté de parties prenantes dont chacune contribue à ce qu'il est et à ce qu'il devient. La question n'est donc pas "qui possède ?" mais "qui est responsable de quoi, et comment ces responsabilités s'articulent-elles en un ensemble cohérent et totalisé ?"

Les parties prenantes comme co-créateurs du produit

La vision conventionnelle des parties prenantes dans le développement logiciel est une vision de consommateurs et de fournisseurs. Les parties prenantes "consomment" le produit ou "fournissent" des exigences, des ressources, des contraintes. Cette vision est fondamentalement passive et transactionnelle. Elle fait des parties prenantes des acteurs extérieurs au produit, qui influencent ce produit de l'extérieur.

La Totalisation inverse cette vision. Selon ŢŢĻŞŢ~, toutes les parties prenantes qui entrent en contact significatif avec un produit logiciel en sont, qu'elles le veuillent ou non, des co-créateurs. L'utilisateur qui utilise le produit chaque jour et qui développe des habitudes autour de lui le façonne aussi sûrement que le développeur qui écrit son code. Le commanditaire qui alloue des ressources de manière inégale influence l'architecture du produit aussi profondément que l'architecte qui la conçoit. La personne du support qui répond aux questions des utilisateurs est une interface vivante entre le produit et sa réalité d'usage.

> *« Co-créateur » ne signifie pas que tout le monde a le même rôle dans la création*

> *du produit. Cela signifie que toutes les parties prenantes participent à la définition de ce que le produit est et devient, par leurs actions, leurs décisions, leurs retours et même leurs silences. Reconnaître cela, c'est la première étape vers une relation totale et réconciliée avec le produit.*
>
> - Philosophie ŢŢĻŞŢ~, ORIGINALIUM

La vision utilisateur intégrée dans le produit dès l'origine

Il y a une pratique que l'on retrouve dans les équipes les plus matures : intégrer la voix de l'utilisateur non pas à la fin du processus, comme une validation, mais au tout début, comme une fondation. Cette pratique n'est pas simplement une bonne technique de développement agile. C'est une posture philosophique qui reconnaît que l'utilisateur est co-créateur du produit dès son origine.

La Totalisation va plus loin que les approches conventionnelles de "centrage sur l'utilisateur" en affirmant que l'utilisateur ne doit pas seulement être "consulté" - il doit être *intégré* dans la vision du produit. Sa réalité quotidienne, ses frustrations, ses aspirations, ses limitations, ses forces, ses contextes d'usage multiples - tout cela devrait habiter l'esprit de chaque personne qui touche au produit, de la conception au déploiement en passant par chaque ligne de code.

Cette intégration n'est pas seulement une affaire de méthodes de recherche utilisateur, aussi importantes soient-elles. C'est une affaire de culture et d'intention. Est-ce que ton équipe pense à l'utilisateur quand elle décide de sa dette technique ? Quand elle choisit entre deux implémentations d'une même fonctionnalité ? Quand elle décide de reporter la correction d'un bug ? La Totalisation te demande d'intégrer l'utilisateur dans chacune de ces décisions - pas comme une contrainte externe, mais comme une présence interne, une conscience qui accompagne chaque geste de création.

Les conflits d'intérêt autour du produit et comment Totalisation les résout

Soyons honnêtes : les parties prenantes d'un produit logiciel n'ont pas toujours les mêmes intérêts. Et ces divergences d'intérêts créent des conflits réels, parfois intenses, qui peuvent fragiliser le produit et les équipes qui le portent. La Totalisation ne prétend pas éliminer ces conflits - elle offre un cadre pour les transformer en forces productives.

Les conflits les plus fréquents autour d'un produit logiciel sont de plusieurs ordres :

- **Le conflit de temporalité :** Les utilisateurs veulent des fonctionnalités maintenant. L'équipe technique veut du temps pour bien faire les choses. Les gestionnaires veulent voir des résultats avant la prochaine réunion de direction.
- **Le conflit de priorité :** Le département A veut que sa fonctionnalité soit développée en premier. Le département B est convaincu que la sienne est plus urgente. Et l'équipe produit essaie de satisfaire tout le monde, finissant souvent par ne satisfaire personne vraiment.
- **Le conflit de vision :** Certaines parties prenantes veulent que le produit soit un outil de productivité interne. D'autres veulent qu'il devienne un produit commercial. D'autres encore rêvent d'une plateforme ouverte à des partenaires externes.
- **Le conflit de qualité :** L'équipe de développement veut prendre le temps de bien faire les choses. La direction veut livrer plus vite. Et quelque part au milieu, la qualité du produit se retrouve otage de cette tension.

La Totalisation résout ces conflits non pas en les tranchant arbitrairement, mais en les élevant à un niveau supérieur. Elle demande à toutes les parties d'abandonner momentanément leurs positions individuelles et de se poser

ensemble la question fondamentale : "Qu'est-ce qui est le mieux pour l'identité et la mission à long terme de ce produit ?" Cette question - quand elle est posée sérieusement et collectivement - a le pouvoir de transformer les conflits en conversations productives, parce qu'elle déplace l'enjeu de "ce que je veux" à "ce que le produit est et devrait devenir".

L'alignement de toutes les parties : une totalisation des perspectives

L'alignement des parties prenantes est souvent présenté comme un objectif de gestion de projet : on organise des réunions, on crée des documents de vision, on fait signer des approbations, et on espère que tout le monde travaille dans la même direction. Mais l'alignement au sens de la Totalisation est quelque chose de bien plus profond et de bien plus vivant que cela.

La totalisation des perspectives, c'est le processus par lequel les différentes visions, expériences et connaissances de toutes les parties prenantes sont réellement intégrées dans la compréhension collective du produit. Pas "consultées", pas "documentées", pas "gérées" - *intégrées*. Chaque perspective enrichit la compréhension globale. Chaque voix, même discordante, révèle quelque chose d'important sur ce que le produit est et sur ce qu'il doit devenir.

Cet alignement total ne signifie pas l'unanimité. Il signifie que chaque partie prenante comprend comment sa perspective s'inscrit dans la vision d'ensemble, même quand ses préférences individuelles ne sont pas pleinement satisfaites. Il signifie que la décision collective est plus riche et plus sage que n'importe quelle décision individuelle aurait pu l'être. C'est le sens profond de l'action d'additionner - synthétiser l'ensemble plutôt que de choisir l'une des parties.

Architecture du produit et vision Totalisation

« L'architecture d'un produit logiciel est son squelette visible - mais aussi le miroir de l'âme de ceux qui l'ont conçu. Une architecture bien pensée raconte une histoire. Une architecture négligée révèle une peur. »

L'architecture comme expression de la vision du produit

Il existe une idée reçue tenace dans le monde du développement logiciel : l'architecture est un problème purement technique, une affaire d'ingénieurs qui optimisent des systèmes, et elle n'a que peu de rapport avec la vision philosophique ou les aspirations du produit. Cette idée est non seulement fausse - elle est dangereuse.

L'architecture d'un produit logiciel est l'expression la plus concrète et la plus durable de la vision qui a présidé à sa création. Chaque décision architecturale est une décision de valeurs. Quand tu décides de construire un système monolithique, tu dis quelque chose sur ce que tu crois être important : la simplicité du déploiement, la facilité de développement, la cohérence du modèle de données. Quand tu optes pour une architecture microservices, tu affirmes d'autres valeurs : l'indépendance des équipes, la résilience aux pannes partielles, la capacité d'évoluer des parties du système sans toucher au tout.

Aucune de ces décisions n'est neutre. Et la Totalisation te demande d'en être conscient, pleinement et honnêtement. Tu ne choisis pas une architecture parce que "c'est ce que font les grandes compagnies" ou parce que "c'est la mode". Tu choisis une architecture parce qu'elle exprime le mieux qui est ton produit et comment il compte servir ceux qui l'utilisent. L'architecture est une déclaration philosophique autant que technique.

La dette technique : une fracture dans la réconciliation

La dette technique est l'un des concepts les plus importants - et les plus mal compris - du développement logiciel. Dans son sens le plus basique, elle désigne l'ensemble des raccourcis, des compromis et des implémentations sous-optimales qui s'accumulent dans un produit au fil du temps. Mais dans la vision de la Totalisation, la dette technique est quelque chose de bien plus significatif : c'est une fracture dans la réconciliation entre le produit et ses créateurs.

Voici pourquoi. Chaque élément de dette technique représente un moment où l'équipe a choisi de faire quelque chose qu'elle savait imparfait plutôt que de prendre le temps de bien faire. Ce choix peut être parfaitement justifié - il y a des situations où la dette technique est une décision stratégique raisonnable. Mais quand cette dette n'est jamais remboursée, quand elle s'accumule sans plan de gestion, elle crée une distance croissante entre ce que le produit devrait être et ce qu'il est réellement.

Cette distance, c'est une fracture de réconciliation. Les développeurs qui travaillent dans un produit fortement endetté techniquement finissent par développer une relation difficile avec ce produit. Ils ont honte de certaines parties de son code. Ils évitent de toucher à certains modules parce qu'ils savent que tout peut s'effondrer. Ils passent plus de temps à gérer la complexité accumulée qu'à créer de la valeur nouvelle. Et graduellement, ils se désolidarisent du produit - ils ne s'y identifient plus, ils ne s'en sentent plus les gardiens.

> *La dette technique non gérée n'est pas seulement un problème d'ingénierie. C'est un problème de réconciliation. Elle crée une distance entre l'équipe et le produit, entre ce que le produit est et ce qu'il pourrait être, entre la fierté du travail bien fait et la honte d'un travail différé. La Totalisation refuse cette distance et demande*

> *de l'affronter avec honnêteté et courage.*
>
> - Philosophie ṰṰḶṢṬ~, ORIGINALIUM

Gérer la dette technique selon ṰṰḶṢṬ~ ne signifie pas l'éliminer à tout prix - ce serait irréaliste. Cela signifie être pleinement conscient de ce qu'elle coûte, la traiter comme une obligation réelle et non comme une abstraction, et s'engager à la rembourser progressivement, de manière planifiée, en la priorisant avec la même rigueur qu'on prioriserait n'importe quelle autre fonctionnalité critique du produit.

La modularité comme principe totalisateur

Parmi tous les principes architecturaux qui peuvent guider la conception d'un produit logiciel, la modularité occupe une place particulière dans la vision ṰṰḶṢṬ~. Et ce n'est pas un hasard : la modularité est, en essence, un principe totalisateur.

Un système modulaire est un système dont les parties sont distinctes mais interdépendantes. Chaque module a une responsabilité claire, des interfaces bien définies, et une certaine autonomie. Et pourtant, les modules ne sont pas des îles - ils forment ensemble un tout cohérent, dont les capacités dépassent la somme des parties individuelles. C'est exactement la définition de la Totalisation : rassembler et réaliser totalement dans son ensemble.

La modularité, bien appliquée, permet quelque chose d'extraordinaire dans un produit logiciel : elle permet à chaque partie d'évoluer à son propre rythme, selon ses propres besoins, sans mettre en danger la cohérence et la stabilité du tout. Elle est l'expression architecturale de la philosophie ORIGINALIUM - chaque élément peut être considéré dans son ensemble propre, tout en restant une partie intégrante du grand ensemble du produit.

La modularité totalisante va cependant au-delà de la modularité technique. Elle s'applique aussi à la façon dont l'équipe est organisée, dont les responsabilités sont réparties, dont les décisions sont prises. Un produit modulaire géré par une organisation monolithique crée des tensions. La vision ŢŢĻŞŢ~ cherche la cohérence entre l'architecture du produit et l'architecture humaine qui le porte.

Comment la vision ŢŢĻŞŢ~ influence les décisions architecturales

Concrètement, que change la philosophie de la Totalisation dans la manière dont on prend des décisions architecturales ? Voici quelques-unes des questions que ŢŢĻŞŢ~ pose systématiquement face à chaque décision architecturale significative :

- **La décision de continuité :** Ce choix architectural permettra-t-il au produit de continuer à évoluer de manière cohérente sur plusieurs années ? Ou est-ce qu'il crée des impasses qui limiteront les options futures ?
- **La décision d'identité :** Cette architecture exprime-t-elle fidèlement ce que le produit est et ce qu'il aspire à être ? Ou est-elle une architecture "par défaut", choisie par habitude ou commodité plutôt que par intention ?
- **La décision de relation :** Comment cette architecture facilitera-t-elle ou compliquera-t-elle le travail des personnes qui vont vivre avec ce produit - développeurs, opérateurs, utilisateurs - dans le temps ?
- **La décision d'ensemble :** Est-ce que cette décision locale s'inscrit harmonieusement dans l'ensemble de l'architecture du produit ? Ou crée-t-elle une incohérence, une exception qui complique la compréhension globale du système ?
- **La décision de coût véritable :** Quel est le coût total de cette décision - pas seulement le coût de mise en oeuvre immédiate, mais le coût de maintenance, d'évolution, et de formation dans le temps ?

L'architecture vivante vs l'architecture figée

Il y a deux manières fondamentalement différentes de concevoir l'architecture d'un produit logiciel, et elles correspondent à deux philosophies de vie radicalement opposées.

La première, que la Totalisation nomme l'architecture figée, est celle qui est conçue une fois, "pour toujours", avec l'intention de définir une structure qui ne changera pas. Cette architecture est souvent présentée comme un gage de sérieux et de rigueur. Elle est documentée en grand détail, validée par de nombreuses personnes, et traitée comme un document de référence immuable. Et puis la réalité arrive, les besoins changent, les technologies évoluent, l'organisation se transforme, et l'architecture figée devient progressivement une cage - une structure qui contraint et ralentit plutôt que d'habilitant et d'accélérer.

La seconde, que ŢŢĻŞŢ~ appelle l'architecture vivante, est celle qui est conçue avec l'humilité de reconnaître qu'elle devra évoluer. Elle est fondée sur des principes stables plutôt que sur des décisions figées. Elle définit clairement ce qui ne changera pas - les valeurs fondamentales du produit, ses engagements envers ses utilisateurs - et laisse délibérément de l'espace pour que le reste s'adapte à mesure que le monde change.

L'architecture vivante n'est pas une architecture sans discipline. Au contraire, elle exige une discipline supérieure : la discipline de distinguer ce qui est essentiel de ce qui est contingent, de protéger le premier tout en laissant le second évoluer librement. C'est exactement ce que fait la Totalisation dans la philosophie ORIGINALIUM : elle identifie ce qui doit être totalisé et protégé à travers le temps, et ce qui peut et doit changer en réponse aux nouvelles réalités.

Principe ŢŢĻŞŢ~ - Architecture vivante

Une architecture vivante est fondée sur des principes permanents et des structures évolutives. Elle reconnaît que le produit traversera des contextes que ses créateurs ne peuvent pas anticiper, et elle est conçue pour accueillir ces contextes avec grâce plutôt qu'avec résistance. C'est une architecture qui exprime la confiance dans la capacité du produit à s'adapter tout en restant lui-même.

CHAPITRE 9

L'identité du produit logiciel

« Le plus grand danger qui menace un produit logiciel n'est pas l'obsolescence technologique. C'est la perte de soi - le moment où le produit ne sait plus qui il est, pourquoi il existe, et pour qui il a été créé. »

Tout produit logiciel a une identité propre

Il peut sembler étrange de parler d'identité à propos d'un logiciel. L'identité, nous dit-on, c'est l'affaire des personnes, peut-être des organisations, mais certainement pas des produits technologiques. Un logiciel, c'est du code, des données, des algorithmes. Comment pourrait-il avoir une identité ?

La philosophie ŢŢĻṢŢ~ répond à cette objection avec une clarté tranquille : tu peux la rejeter, mais tu ne peux pas nier l'expérience qui la sous-tend. Tu as déjà utilisé un logiciel et senti immédiatement que "c'est un produit qui te comprend, qui te respecte, qui sait ce qu'il veut être". Et tu as utilisé d'autres produits qui t'ont donné la sensation inverse : confus, hésitants, qui essaient d'être tout pour tout le monde et finissent par n'être rien pour personne. Cette différence que tu

ressens, c'est la différence entre un produit qui a une identité forte et un produit qui en manque.

L'identité d'un produit logiciel est l'ensemble cohérent de ses valeurs fondamentales, de sa mission, de sa manière unique de résoudre les problèmes, de la relation qu'il entretient avec ses utilisateurs et de l'empreinte distinctive qu'il laisse dans la vie de ceux qui l'utilisent. C'est ce qui fait qu'on reconnaît un produit immédiatement, même sans voir son logo - parce que sa manière d'être est reconnaissable, distinctive, unique.

L'âme du logiciel : ce qui le rend unique et précieux

L'âme d'un produit logiciel - et la Totalisation n'a pas peur de ce mot - c'est l'ensemble des intentions fondamentales qui ont présidé à sa création et qui continuent de l'animer à travers toutes ses évolutions. Ce sont les convictions profondes de ceux qui l'ont créé, qui se sont cristallisées dans des choix de conception, des priorités d'évolution, des refus catégoriques de certaines fonctionnalités, des manières particulières d'organiser l'expérience utilisateur.

L'âme d'un produit se révèle souvent dans ce que le produit refuse de faire autant que dans ce qu'il fait. Un produit qui a une âme forte sait dire non. Il sait qu'il y a des fonctionnalités qu'il ne développera pas, des marchés qu'il ne cherchera pas à conquérir, des compromis qu'il ne fera pas - non pas parce qu'il en est incapable techniquement, mais parce que ces choix trahiraient ce qu'il est fondamentalement.

Cette capacité à dire non est l'une des expressions les plus précieuses de l'identité d'un produit. Et c'est aussi l'une des plus difficiles à protéger dans les organisations, où la pression de "toujours faire plus, pour toujours plus de gens, dans toujours plus de contextes" est constante et intense. La Totalisation est en partie l'art de savoir quels non valent la peine d'être maintenus, même sous pression.

> *Ce qui rend un produit logiciel vraiment précieux, ce n'est pas la quantité de fonctionnalités qu'il offre. C'est la profondeur avec laquelle il accomplit ce pour quoi il a été créé - et la clarté avec laquelle il sait ce qu'il est et ce qu'il n'est pas. La préciosité vient de la cohérence, pas de la complétude.*
>
> - Philosophie ŢŢĻŞŢ~, ORIGINALIUM

Comment perdre l'identité d'un produit (et pourquoi ça arrive)

La perte d'identité d'un produit logiciel est l'un des phénomènes les plus communs et les plus désolants de l'industrie technologique. Elle n'arrive presque jamais d'un coup. Elle est le résultat d'une série de petites décisions, prises individuellement pour de bonnes raisons, qui s'accumulent jusqu'à transformer le produit en quelque chose d'irréconnaissable.

Voici les chemins les plus courants qui mènent à la perte d'identité :

- **La dérive fonctionnelle :** On ajoute des fonctionnalités en réponse à chaque demande de chaque partie prenante, sans se demander si ces fonctionnalités s'inscrivent dans la mission centrale du produit. Le produit grandit en volume, mais se rétrécit en clarté. Il devient un agrégat de fonctionnalités sans fil conducteur.
- **La rotation des équipes :** Les personnes qui portaient la vision et la mémoire du produit partent. Les nouveaux arrivants ne connaissent pas l'histoire, ne comprennent pas le "pourquoi" de certaines décisions, et commencent à faire des choix qui semblent raisonnables isolément mais qui s'éloignent progressivement de l'identité originale.
- **La pression concurrentielle :** Un concurrent sort une fonctionnalité populaire, et la réponse réflexe est de la copier. Puis une autre. Et encore

une autre. Jusqu'à ce que le produit ait perdu toute originalité et ressemble à tous les autres, sans être meilleur qu'aucun.

- **Le changement de marché cible :** L'organisation décide de cibler un nouveau segment de clientèle, plus lucratif, qui a des besoins différents. Le produit est modifié pour ce nouveau public. Et le public original, celui pour qui le produit avait été créé, se retrouve avec quelque chose qui ne lui ressemble plus.
- **Les décisions technologiques opportunistes :** On refactorise le produit pour adopter une nouvelle technologie "parce que c'est la mode" ou "parce qu'on a du mal à recruter avec les anciennes technologies". Et dans le processus, des comportements subtils mais importants du produit sont perdus, parce que personne n'avait pris le temps de les documenter ou de les comprendre assez profondément pour les préserver.

Ce que tous ces chemins ont en commun, c'est l'absence de gardien conscient de l'identité du produit. En l'absence de quelqu'un - ou d'un processus - qui questionne chaque décision importante à l'aune de l'identité du produit, cette identité s'érode naturellement. L'érosion est l'état naturel. C'est la préservation de l'identité qui demande un effort conscient et continu.

Totalisation comme gardienne de l'identité du produit

C'est ici que la philosophie ŢŢĻṢŢ~ révèle l'une de ses fonctions les plus essentielles : celle de gardienne de l'identité du produit. Mais qu'est-ce que cela signifie concrètement ? Comment la Totalisation remplit-elle ce rôle ?

Premièrement, la Totalisation demande que l'identité du produit soit explicitement définie et documentée, non pas comme un exercice de marketing, mais comme un acte philosophique fondateur. Qui est ce produit ? Quelles sont ses valeurs fondamentales ? Quelle est sa mission centrale - non pas le problème technique qu'il résout, mais le besoin humain profond auquel il répond ? Quels

sont ses engagements envers ses utilisateurs, ses limites acceptées et ses refus catégoriques ? Ces réponses forment le socle identitaire du produit, le point de référence auquel toutes les décisions futures seront confrontées.

Deuxièmement, la Totalisation intègre cette référence identitaire dans le processus de décision quotidien. Chaque fonctionnalité envisagée est évaluée : "Est-ce que cela enrichit ou dilue l'identité de ce produit ?" Chaque décision architecturale est questionnée : "Est-ce que cette structure exprime ou trahit ce que ce produit est fondamentalement ?" Chaque compromis est pesé : "Quel est le coût identitaire de ce compromis, et est-il acceptable au regard de la mission du produit ?"

Troisièmement, la Totalisation s'assure que l'identité du produit est transmise activement aux nouveaux membres de l'équipe, non pas comme une liste de règles à respecter, mais comme une histoire à comprendre et à continuer. La transmission de l'âme d'un produit est un acte culturel, pas un acte de formation technique. Elle demande que les anciens partagent non seulement ce que le produit fait, mais pourquoi il le fait de cette manière-là, et ce que cela dit de ce qu'il est.

TTLST~ - La gardienne identitaire en action : quatre postures

Posture	Question centrale	Fréquence
La mémoire	D'où vient ce produit et qu'est-ce qui a toujours été vrai à son sujet ?	En continu - chaque nouvelle décision

La boussole

Cette décision nous rapproche-t-elle ou nous éloigne-t-elle de ce que ce produit doit être ?

À chaque carrefour décisionnel

Le miroir

Ce que nous faisons reflète-t-il fidèlement ce que nous disons être ?

Périodiquement - révision d'identité

La voix

Si ce produit pouvait parler, que dirait-il de la décision que nous nous apprêtons à prendre ?

Lors des décisions à fort impact identitaire

Le produit comme oeuvre collective portant des valeurs

Il y a une dernière dimension de l'identité du produit logiciel que la Totalisation tient à honorer, et c'est peut-être la plus belle et la plus importante : un produit logiciel est une oeuvre collective. Il porte, inscrites dans chaque ligne de code, chaque décision de conception, chaque choix d'interface, les empreintes de toutes les personnes qui l'ont touché. C'est une création humaine au sens le plus riche du terme.

Et comme toute oeuvre humaine, un produit logiciel peut être porteur de valeurs. Il peut être conçu avec l'intention d'honorer la dignité de ses utilisateurs, de faciliter des connexions humaines authentiques, de simplifier ce qui était inutilement complexe, de démocratiser l'accès à des ressources ou des capacités qui n'étaient disponibles qu'à certains. Ces valeurs ne sont pas accidentelles -

elles sont le fruit de choix conscients, faits par des personnes qui ont décidé que leur travail avait un sens au-delà de la complétion d'un backlog.

La Totalisation te demande de prendre au sérieux cette dimension. De te demander non seulement "Qu'est-ce que je construis ?" mais aussi "Quelles valeurs est-ce que j'inscris dans ce que je construis ?" et "De quelle manière est-ce que ma contribution - même la plus technique, même la plus invisible - participe à l'expression de ces valeurs dans le monde ?"

Un développeur qui écrit du code maintenable et lisible dit quelque chose à ses collègues futurs : "Je te respecte. Je pense à toi, même si je ne te connais pas encore." Un architecte qui conçoit un système qui peut évoluer gracieusement dit quelque chose à l'avenir : "Je fais confiance à ce que vous deviendrez." Un concepteur qui prend le temps de comprendre vraiment les utilisateurs avant de dessiner une interface dit quelque chose aux personnes qu'il sert : "Ton temps et ton intelligence ont de la valeur pour moi."

Ces gestes, multipliés par toutes les personnes qui touchent à un produit, créent quelque chose qui dépasse la somme de leurs parties individuelles. Ils créent une oeuvre collective dotée d'une âme, d'une identité, d'une capacité à transmettre des valeurs à travers le temps et à travers les interactions humaines qu'elle rend possibles. C'est à cette ambition que la Totalisation te convie - non pas seulement construire du logiciel, mais participer à la création de quelque chose qui porte du sens.

> *Quand tu quittes un projet, ce que tu laisses derrière toi n'est pas seulement du code. C'est la trace de tes valeurs, de ton attention, de ton respect pour le travail bien fait et pour les humains qui vivront avec ce que tu as créé. La Totalisation te demande de laisser cette trace avec intention et avec fierté.*

- Philosophie ŢŢĻŞŢ~, ORIGINALIUM

Note de fin de Partie B - Vers la Partie C

Tu viens de traverser cinq chapitres qui ont approfondi considérablement ta compréhension de ce que signifie vivre avec un produit logiciel sous le regard de la Totalisation. Tu as exploré le cycle de vie du produit non pas comme une séquence d'étapes techniques, mais comme l'existence complète d'un être qui naît d'un ressenti humain et qui mérite d'être accompagné avec intention à chaque phase de sa vie. Tu as réexaminé la qualité - non plus comme une liste de critères à cocher, mais comme une intention fondatrice qui traverse tout. Tu as questionné la notion de propriété du produit pour la remplacer par une vision plus riche et plus vraie de co-création collective. Tu as vu comment l'architecture est une déclaration philosophique autant que technique, et comment la Totalisation transforme chaque décision architecturale en un acte de fidélité à l'identité du produit. Enfin, tu as rencontré cette idée peut-être la plus transformatrice de toutes : un produit logiciel a une identité propre, une âme, et la préserver est l'un des actes les plus importants et les plus nobles que tu peux accomplir dans ton travail.

La Partie C t'attend avec trois nouveaux territoires d'exploration, encore plus profonds et encore plus décisifs.

Aperçu de la Partie C :

- **Chapitre 10 - La gouvernance du produit logiciel selon ŢŢĻŞŢ~ :** Comment les décisions sont-elles prises autour d'un produit de

manière totalisante ? Quels mécanismes de gouvernance honorent la complexité humaine et technique du produit sans la trahir par des raccourcis managériaux ?

- **Chapitre 11 - L'équipe comme organisme vivant au service du produit :** L'équipe n'est pas un ensemble de ressources - c'est un organisme qui vit et évolue en symbiose avec le produit qu'elle porte. La Totalisation examine cette relation organique et propose une vision radicalement différente de ce que signifie "faire équipe" autour d'un produit logiciel.
- **Chapitre 12 - La réconciliation finale : quand le créateur retrouve son oeuvre :** Le chapitre de synthèse. Après avoir exploré toutes les dimensions du produit logiciel à travers le prisme de la Totalisation, ce chapitre t'amènera à vivre l'expérience de la réconciliation réelle - ce moment où tu regardes ce que tu as créé, ou ce que tu continues de créer, et où tu te dis : "Je me reconnais dans ce produit. Il est moi, et je suis lui. Nous sommes réconciliés."

La Partie C sera la conclusion de ce manifeste. Elle rassemblera en un tout synthétique et totalisé tout ce que les parties A et B auront semé. Continue. Tu es proche du coeur du sujet.

- T̥T̥L̥S̥T~ -

RASSEMBLER ET RÉALISER TOTALEMENT DANS SON ENSEMBLE - AVANT, PENDANT ET APRÈS - EN CONTINU, SANS INTERRUPTION

© Léo Michel Bélanger - Tous droits réservés

Images et couverture par Soléane Bélanger

Publié par ORIGINALIUM | T̥T̥L̥S̥T~

Série ORIGINALIUM - Partie B